

Erlang in ACL2: A Continuation-Passing Style Evaluator

Kuter Bora

Department of Computer Science
Max Plank Institute
Bochum, Germany
kuter.bora@mpi-sp.org

Mark R. Greenstreet

Department of Computer Science
The University of British Columbia
Vancouver, Canada
mrg@cs.ubc.ca

We present an evaluator in ACL2 for Erlang, a functional language for concurrent programming. The evaluator is written in two parts: an AST evaluator and a scheduler. The AST evaluator uses continuation passing style. At each point in an execution, each process has a current continuation that holds all state needed for further execution of the process. Thus, the global state of an Erlang program is a mapping from Erlang process identifiers to their continuations. The scheduler provides a non-deterministic interleaving of processes between globally visible actions such as process spawns and message sends and receives. We demonstrate the evaluator by proving some properties of a few, simple Erlang programs.

1 Introduction

“Erlang is a concurrent language consisting of communicating components where the components are written in a functional language.” [4]. Erlang supports concurrent computing by providing lightweight processes that communicate through message passing. Erlang has been widely used, especially for telecommunications applications – see <https://erlang-companies.org>.

There are many static analysis tools for Erlang including Dialyzer [18], a linter; Typer [15], a type annotation tool; and Wrangler [14], a refactoring tool. The only other theorem-proving approach to verification of Erlang programs that we are aware of is the Erlang Verification Tool (EVT) [11] is a language specific (i.e. Erlang) proof-assistant based on the μ -calculus.

Using continuations with ACL2 has several motivations. First, continuations provide a unified approach for modeling concurrency, communication, and exceptions. Second, ACL2 offers a high degree of proof automation. These two properties appear to be a good fit for the reasoning about Erlang programs. Erlang is often used for systems with stringent reliability requirements including fault tolerance. Being able to formally verify such software can contribute to reliability. Like ACL2, Erlang is an untyped, functional language – there is a natural semantic match. Finally, Erlang is used in industrial development environments including dynamic updates to running code. There is a need to be able to quickly re-verify systems when the code is changed; thus, verification is not a once-and-done, but a process that needs to be included in regression tests, an area where ACL2 excels.

This paper presents our initial foray into using ACL2 and continuations to reason about Erlang programs. Our contributions are:

- An Erlang to ACL2 translator. We use the front-end of the Erlang compiler to generate abstract syntax trees that are then output in ACL2 syntax for use by our evaluator.
- A continuation passing style evaluator for these ASTs.
- Two examples to demonstrate that our method works end-to-end: showing that a simple, recursive function for computing the sum of natural numbers up to N returns the same value as the

closed-form formula; and a parallel implementation of reduce. These examples both show that our evaluator works both to run Erlang code and to provide a semantics for reasoning about it. They also point to the next steps for this line of research.

2 Background and Related Work

2.1 Erlang

Erlang is a language for concurrent programming using message passing where individual processes are defined functionally [4]. In other words its a functional language extended with operations for spawning processes and sending and receiving messages. For example to compute the sum of the first `n` natural numbers, one can write:

```
sum_of_nats(0) -> 0;
sum_of_nats(N) when is_integer(N), 0 < N ->
  N + sum_of_nats(N-1).
```

Erlang makes extensive use of pattern matching. Functions are defined as a list of clauses separated by semicolons. Each clause has a pattern for each argument. In the definition of `sum_of_nats` above, there are two clauses: `sum_of_nats(0)` matches when the value of `N` is 0; and `sum_of_nats(N) when ...` matches when the value of `N` is a positive integer. If `sum_of_nats` is called with an argument that is not a natural number, then it throws a `function_clause` exception

The spawn function in Erlang, `spawn(F)` creates a new process, returns the Erlang pid for the process to the caller of spawn, and executes `F` in the child process. `F` is a function of no arguments – data can be passed to the child process by lambda capture. When `F` returns, the return value is discarded and the child process exits. There are several variants on the spawn call that take additional arguments; see the [Erlang documentation](#) for more details.

Messages are sent with a sent expression:

```
DestinationPid ! ValueToBeSent
```

Receive expressions have the form:

```
receive
  Pattern1 -> Expr1;
  Pattern2 -> Expr2;
  ...
end
```

where each expression can use the values of variables bound by the corresponding pattern. Message delivery is reliable as with TCP-IP. For any two (not necessarily distinct) pids, `pid1` and `pid2`, Erlang guarantees the messages sent from `pid1` to `pid2` are received at `pid2` in the order in which they were sent from `pid1`. Erlang provides no other ordering guarantees.

Figure 1 shows a simple example of Erlang processes. First, we will decode the syntax for readers who are new to Erlang. In Erlang, identifiers that start with a capital letter are variables and those that start with a lower case letters are atoms; an Erlang atom is analogous to a lisp symbol. An expression of the form

```
Pattern = Expression
```

is a pattern match. If the pattern matches, free variables in `Pattern` will be bound to value of the term in `Expression` to which they matched. The pattern `[X, Y | Rest]` matches any list with two or more elements, binding `X` and `Y` to the first two elements and `Rest` to the possibly empty remainder of the list. If a variable in `Pattern` is already bound to a value, then the corresponding term in `Expression` must

```

add_pairs(NumberList) ->
  WorkerPid = spawn(fun() -> server(none) end),
  Result = helper(WorkerPid, NumberList),
  WorkerPid ! exit,
  Result.

server(Left) -> % a simple reactive process
  receive % receive two numbers and send their sum back to the requester
    X when (Left == none), is_number(X) ->
      server(X);
    {Requester, Right} when is_number(Right) ->
      Requester ! Left + Right,
      server(none);
  exit -> exit
end.

helper(WorkerPid, [X, Y | Rest]) ->
  WorkerPid ! X,
  WorkerPid ! {self(), Y},
  receive Sum -> [Sum | helper(WorkerPid, Rest)] end;
helper(_, []) -> []. % empty list, nothing to do

```

Figure 1: An example of Erlang Processes

have the same value. A comma-separated sequence of expressions is like a lisp `progn`. The value of the list of expressions is value expression in the sequence. An Erlang tuple can be written as `{A, B, C}`; tuples are often used to represent a fixed number of heterogeneous data values. An Erlang lambda expression is written as

```

fun(ArgsPattern1) -> Body1;
  (ArgsPattern2) -> Body2;
  ...
end

```

The function `add_pairs(NumberList)` takes a list (of even length) of numbers, treats it as a list of pairs, and returns a list of the sums of each pair. For example,

```
add_pairs([1,2,3,4,5,6,7,8]) -> [3,7,11,15].
```

The function `server` implements a reactive process. It waits to receive a request, handles the request, and unless the request is `exit`, it makes a recursive call to await the next request. By design, the server function can handle an arbitrary number of requests without exiting. We can't prove termination of `server` because it may not terminate. We address this in Section 3.3. The `server` function is tail-recursive to avoid stack overflow.

Spawn, send, receive, and exit are not applicative. For example, the ordering of operations by different processes is non-deterministic, as is the ordering of message delivery for messages from different processes. Consider 2. We can evaluate

```
add_pairs_nd(lists:seq(1, 1000000)) == add_pairs_nd(lists:seq(1, 1000000)).
```

and find that it often evaluates to `false`. This clearly violates referential transparency. This is because the order in which `server` receives messages from each of the two processes calling `helper` is unspecified.

```

add_pairs_nd(NumberList) ->
  MyPid = self(),
  List1, List2 = lists:split(length(NumberList) div 2, NumberList),
  WorkerPid = spawn(fun() -> server(none) end),
  spawn(fun() -> MyPid ! helper(WorkerPid, List1) end),
  spawn(fun() -> MyPid ! helper(WorkerPid, List2) end),
  Result = receive Sum1 -> Sum1 end ++ receive Sum2 -> Sum2 end,
  WorkerPid ! exit,
  Result.

```

Figure 2: Adding pairs in parallel.

While Erlang processes are non-applicative, they provide a simple and elegant logical story for files and other operating services. To model processes and their related operations in our evaluator, we use continuations as described below.

For a more complete introduction to Erlang see [10, 13, 6].

2.2 Continuation Passing Style

Continuation Passing Style (CPS) provides a very general method for describing a computations including concurrent processes, sending and receiving of messages, and exceptions as well as expression evaluation, control flow, and assignments. A continuation is a function that says “what to do next” [3, p. 2]. In a CPS program, each continuation, **C**, performs a single action to get a new value and calls the next continuation with that value. A terminal continuation just returns the value with which it was called.

As an example, Figure 3 shows a CPS version of the `sum_of_nats` function. Functions **C1** through **C5** are continuations. To see how CPS “works”, consider execution of `sum_of_nats_cps(N)` when $N=0$, when N is a positive integer, and when N is not a natural number.

Each continuation is called with the value computed by the previously executed continuation, performs an operation – i.e. it updates `Val` – then it calls the next continuation. The actual parameter for each call is either a constant or a single variable, no complicated expressions. This makes control flow explicit. Furthermore, each continuation has all of the data needed for the rest of the computation. In Figure 3, results from previous continuations that are needed in the future are part of each function’s closure. Of course, ACL2 does not support higher-order functions. Section 3.2 describes our implementation.

Continuations provide a unified way to model concurrency, communication, and exceptions, as well as modeling “ordinary” sequential computation. Because a continuation holds all of the information needed for the rest of the computation, we can represent a collection of concurrently executing processes as a mapping from each pid to its current continuation. When a process is preempted or blocked, its current continuation is simply returned to the scheduler. When the process is run again, it just resumes at that continuation. Exceptions are also simple – if an operation throws an exception, the value that it passes to the next continuation is tagged as being an exception. A continuation for an exception handler choose from a set of possible successors depending on whether or not an exception was encountered, and if so, whether or not this continuation has a handler that matches the exception. If a continuation cannot handle an exception, it just passes it to its successor.

```

sum_of_nats_cps(N, C0) ->
  C1 = fun(true) -> C0(0); % N == 0
  (false) -> % N /= 0
    C2 = fun(true) -> % is_integer(N) and (0 < N)
      C3 = fun(Sum_N_minus_1) ->
        Val = N + Sum_N_minus_1,
        C0(Val)
      end,
      Val = N-1,
      sum_of_nats_cps(Val, C3)
    end,
    C4 = fun(N_is_integer) ->
      C5 = fun(N_is_pos) ->
        Val = N_is_integer and N_is_pos,
        C2(Val)
      end,
      Val = 0 < N,
      C5(Val)
    end,
    Val = is_integer(N),
    C4(Val)
  end,
  Val = N == 0,
  C1(Val).

```

Figure 3: Sum of the natural numbers up to N in Continuation Passing Style

2.3 Related Work

The most closely work that we know of is Turán *et al.* [21] that derives an interpreter for Core Erlang [8]. For historical and pragmatic reasons, Erlang has a significant amount of syntactic sugar. While Core Erlang is a desugared variant of Erlang to which Erlang code can be readily translated. The goal of Turán *et al.* is roughly the dual of what we have done: given an operational semantics for Core Erlang [7], they automatically generate an interpreter in Haskell; in contrast, we present an evaluator for Erlang which gives it a semantics for reasoning about Erlang programs in ACL2.

Using evaluators to define the semantics of a language is common practice in ACL2 including the JVM [16], Verilog [20], and the x86 instruction set [12]. We believe that our current work is the first published continuation passing style evaluator in ACL2.

3 The CPS Evaluator

The Erlang front end leverages the components of the Erlang compiler that are provided by the standard Erlang library, OTP. The scanner, `erl_scan:string(String)`, attempts to tokenize `String` returning a list of tokens if successful. The parsers, `erl_parse:parse_exprs(Tokens)` and `erl_parse:parse_form`, attempt to parse a list of tokens for list of Erlang expressions or for an Erlang term respectively, returning the corresponding ASTs if successful. Forms include module attributes and function definitions. In short, expressions are Erlang code that can be evaluated, while forms define entities such as functions in the logical world. For example: `erl_parse_exprs` of `"2 + 2."` becomes

```
{:op, _, '+', {:integer, _, 2}, {:integer, _, 2}}.
```

Having parsed Erlang into an AST, we then define a fixtype in ACL2 that models it. The Erlang and ACL2 ASTs are almost one-to-one equivalent: for example, the ACL2 equivalent for `"2 + 2."` is `(:binop (:integer 2) (:integer 2))`. There are two exceptions. First, `erl_parse` represents some lists (function arguments, tuple elements) as Erlang lists, `[ast1, ast2 ...]` rather than `{cons, ast1, {cons, ast2, ...}}`. We normalize the ASTs to match the latter case. Second, `erl_parse` represents all function calls with the `call` keyword. We can statically figure out the type of a `call` expression, so, for ease of implementation, we tag each `call` as a local, remote, or fun call. Definitions of modules and function definitions closely follow the Erlang equivalent. In our ACL2 representation, an Erlang module is a `defprod` containing its attributes and a map from function name to clauses. An Erlang logical world is defined as a map from module names to modules.

The module `erl_to_acl2.erl` provides functionality for parsing Erlang expressions and modules to their ACL2 equivalents. Our current implementation does not support importing or exporting export functions. Until support is added, module attributes of external functions must be explicitly given by the user manually, i.e. by writing `mod:f(X, Y, Z)` instead of

```
-import(mod, [f/3]).
```

```
...
```

```
f(X, Y, Z)
```

The ACL2 evaluator consists of three parts: an AST evaluator that interprets an AST within the context of a single Erlang process, a scheduler that interleaves the computations of multiple processes and conveys messages between processes, and a logical world. The logical world provides a mapping from function names to their lisp AST's. Currently, we assume that all functions are in the same module. Support for multiple modules is a short-term ToDo.

The remainder of this section describes this design in more detail.

3.1 Processes, Scheduling, and Traces

We model the execution of an Erlang program as a trace [9, 17]. A trace is a sequence of states where each state is a mapping from process identifiers to the current continuation list for that process. As described later in this section, the process state also includes an inbox and outbox for messages and a logical-world that maps function names to their ASTs. When the scheduler takes an action, it can deliver a pending message to a process, or it can continue the execution of a process. When a process executes, the evaluator interprets its continuations until either the process exits or until it performs a `receive` operation. When the AST evaluator encounters a `receive` operation, it returns the current continuation (that starts with a `receive` operation) to the scheduler. This continuation includes per-destination lists of messages that were sent during this execution segment. There is a separate message-send queue for each destination pid to implement Erlang’s message ordering guarantees. Erlang provides no other ordering guarantees for message delivery.

To model non-determinism, we describe the scheduler as a constrained function using an encapsulation. In the present implementation, the only fairness condition that we ensure is that if there is a non-stuttering action available, the scheduler will choose some non-stuttering action. Theorems about Erlang programs must hold for all possible schedules that satisfy this very weak condition.

We plan to strengthen this to require weakly fair scheduling: if a process is runnable, then it will eventually be selected to run, or the process will be terminated. Likewise, if a message is pending delivery, then it will eventually be selected to be delivered, or the destination process will exit. Erlang semantics have no notion of an action being intermittently enabled; thus, there is no difference between weak and strong fairness.

Using `defattach` one can provide an executable scheduler for the abstract version described above. This allows our evaluator to be used both to execute Erlang programs and to reason about them.

3.2 The AST evaluator

Within a single process, Erlang is an almost purely functional language. As described in Section 2.1, referential transparency is trivially violated by message passing. To model execution within a single process, we use continuation passing style evaluation of an abstract syntax tree (AST). Figure 3 implemented continuations using closures. Our evaluator constructs a list of these continuations, where each continuation is a tag-sum data structure [2]. Each continuation performs a single primitive Erlang operation, and then the next continuation in the list is evaluated with the result. For each class of Erlang operations – e.g. binary operations, unary operations, case expressions, receive expressions, function calls, etc. – the product structure (see [1]) that has fields according to the data needed by the operation and a next-continuation field. Figure 4 illustrates this by showing a somewhat simplified evaluation of the AST corresponding to

```
(X + Y) * (receive Z -> Z end).
```

A few explanatory remarks about the code. The AST to be evaluated comes from the Erlang compiler front-end as described above. For clarity and succinctness, we write `(ast QuotedExpression)` to denote the AST for `QuotedExpression`. Likewise, the evaluator has a tag-sum data structure to represent Erlang data values, but we write, for example, `(* C5.left RightValue)`, as if these values were their lisp counterparts, again for clarity and succinctness. The functions `cont-expr`, `cont-binop1`, etc. are continuation constructors for the `expr` variant, the `binop1` variant, etc.

The continuation list of each process is paired with an `estate`, (Erlang state) which stores aspects of the program state produced by the previous continuation, including the latest value retrieved by executing

```

ContTail = whatever follows the evaluation of (X + Y) * (receive Z -> Z end).
C1 = (cont-expr :expr (ast '((X + Y) * (receive Z -> Z end)))).
Initial continuation list: (list* C1 ContTail).

eval_cont_list((list* C1 ContTail) EState) → ; C1.kind = 'expr
  (let ((C2 (cont-expr :expr (ast '(X + Y))))
        (C3 (cont-binop1 :op '*'
                          :right_expr (ast '(receive Z -> Z end))
                          (NewState (update-estate->val Estate 'none))))
    eval_cont_list((list* C2 C3 ContTail) NewState)).

eval_cont_list((list* C2 C3 ContTail) Estate) → ; C2.kind = 'expr
  ; Evaluating C2 may produce a value directly, or it may prepend additional continuations
  ; to (list* ContTail) and continue execution from there. Eventually (assuming the evaluator
  ; does not run out of fuel), we will reach a state where the pending continuation list is (list* C3 ContTail)
  ; and the val field of NewState is the value from evaluating (ast '(X + Y)). We skip the
  ; details and continue after the evaluation of C2 and its children is complete.
  (let ((NewState (as described in the comments above)))
    eval_cont_list((list* C3 ContTail) NewState)).

eval_cont_list(C3 | ContTail EState) → ; C3.tag = 'binop1
  (let ((C4 (cont-expr :expr (ast '(receive Z => Z end))))
        (C5 (cont-binop2 :op '*'
                          :left (estate->val Estate)))
    (NewState (update-estate->val Estate 'none)))
    eval_cont_list((list* C4 C5 ContTail) NewState)).

eval_cont_list((list* C4 C5 | ContTail) Estate) → ; C4.tag = 'expr
  (let ((C6 (cont-receive :clauses (list (ast '(Z -> Z)))))
    (NewState (update-estate->val Estate 'none)))
    eval_cont_list((list* C6 C5 ContTail) NewState)).

eval_cont_list([(list* C6 C5 ContTail) Estate] → ; C6.tag = 'receive
  (let ((RecState
        (update-estate->val Estate
          ('receive :klst (list* C5 ContTail)
                    :clauses C6.clauses)))
    ; yield to the scheduler by returning the continuation list, wrapped inside the state
    RecState)).

; when the scheduler runs this process again, it makes the following call to eval_cont_list.
eval_cont_list((list* C7 C5 ContTail) EState) ->; C7.tag = 'receive2
  if there is a message in inbox matching X
    Dequeue the matching message binding the val field of NewState to the value of that message.
    eval_cont_list((list* C5 ContTail) NewState)
  else
    EState ; No matching message. Yield to the scheduler again.
  end.

eval_cont_list((list* C5 ContTail) Estate) → ; C5.tag = 'binop2
  (let ((ExprVal (case-match C5.op
                    ...
                    ...))) ('* (* C5.left (estate->val Estate))
    eval_cont_list(ContTail (update-estate->val Estate ExprVal)).

```

Figure 4: Evaluating $(X + Y) * (\text{receive } X \rightarrow X \text{ end})$

an expression, current bindings of variable names to values, module and logical world, as well as the 'out-box' to be handed off to the scheduler. Each process also has a corresponding 'in-box' maintained by the scheduler. The "world" is a map from names to modules, where the module is a `defprod` containing the attributes of a module and a map from function names to their lisp AST clauses.

To evaluate a binary operation such as the multiplication in `(X + Y) * (receive Z -> Z end)`, the evaluator creates two continuations. The first, `C2` in Figure 4, is a continuation to evaluate `X + Y`. The second, `C3`, is a `binop1` continuation that has fields for the operation to be performed and the AST for the right operand. Evaluation of a `binop1` continuation receives the value for the left operand from the previous continuation (i.e. `C2`) and constructs two more continuations `C4` and `C5`. Continuation `C4` evaluates the right operand. Evaluation `C5` receives the value for the right operand. The operator and the value for the left operand are stored in the `op` and `left` fields respectively of `C5`. `C5` performs the operation and provides the result to the next continuation.

The `receive` operation shows how the AST evaluator interacts with the scheduler. When the `receive` is first encountered, (`C6` in the example), the evaluator wraps the remaining continuations along with the current `estate` in `RecState` and yields to the scheduler.

All process state is contained in `RecState`. The `val` field holds the value generated by the previous continuation, possibly `'none'`. When the scheduler runs this process again, it simply calls `eval_cont_list` on the continuation list for the process obtained from `RecState`. This starts with a `receive2` operation that checks to see if there is a suitable message in the processes' inbox. If so, the value from that message is provided to the next continuation in the list. Otherwise, the process yields to the scheduler again to wait for new messages to arrive.

3.3 Fuel

For our evaluator to be admissible in ACL2, we must show that it always terminates. We give up immediately. The most common use-case for Erlang is to implement reactive processes. In the simplest form, the process waits to receive a request, processes the request, and then calls itself to be ready for the next request. By design, such a process *never terminates*. Thus, attempts to prove termination are doomed to failure.

Instead of proving termination, the continuation structure includes a `fuel` field, where `(natp fuel)`. The evaluator decrements `fuel` for each new continuation, and exits with an out-of-fuel status if `(zp fuel)`. In Section 4 we show how we can prove interesting properties of programs by showing that there is an amount of fuels that is sufficient to complete specific tasks.

4 Examples

We show two examples of proving properties of Erlang programs using our evaluator.

1. Show that the sum of the natural number up to N is $\frac{N^2+N}{2}$.
2. Show that Schwartz's parallel algorithm for reduce [19] computes the sum of the values contributed by each worker process.

The code for the Erlang front-end, our evaluator, and the examples in this section is on github at

<https://github.com/KuterBora/Erlang-in-ACL2>

4.1 `sum_of_nats`

We show that if `N` is a natural number, then

$$\text{sum_of_nats}(N) = \frac{1}{2}(N^2 + N) \quad (1)$$

The code is in the book `Erlang-in-ACL2/books/erl-arithmetic/sum_of_nats.lisp`.

Our proof strategy is to introduce the corresponding function, `sum-of-nats` in lisp, and prove if `N` is the Erlang representation of a natural number, then:

```
sum_of_nats(N) =
  (make-erl-val-integer :val (sum-of-nats (erl-val-integer->val N)))
```

In other words, the result of applying `sum_of_nats` to `N` is the same as converting `N` from our representation of an Erlang integer to a lisp integer, applying `sum-of-nats` to that integer, and converting the result back to an Erlang integer. The key functions and lemmas are:

`sum-of-nats`: a function, the `sum_of_nats` function in ACL2 lisp.

`sum-test-w`: a function that creates an Erlang logical world where `sum_of_nats` is defined. This function exists because our Erlang-to-AST-in-Lisp function originally just printed the AST. We did a cut-and-paste to put it in this book. We are in the process of making the Erlang AST translator write an ACL2 book that can be certified. Then we will be able to load the AST from that book.

`apply-k-of-sum-induct`: In the current code, the function called `eval-cont-list` is called `apply-k`. It will almost certainly get renamed to `eval-cont-list` to allow for a consistent naming convention. `apply-k-of-sum-induct` does a beta-reduction of `sum_of_nats(N)` for the case when `N` is a positive integer.

`apply-k-of-sum-base`: Same as above for the case when `N=0`.

`inductive-step-is-wf`: We say that the value produced by a continuation is well-formed if it isn't an exception and it doesn't represent an out-of-fuel condition for the evaluator. This theorem says that if `N` is a positive integer (i.e. the recursive case) and `sum_of_nats(N)` is well-formed, then the value produced by the recursive call, `sum_of_nats(N-1)` is also well-formed.

`inductive-step-up-to-next-recursive-call`:

Rewrites `sum_of_nats(N)` to `N + sum_of_nats(N-1)` for the case when `N` is a positive integer. The theorem accounts for the evaluator running out of fuel. If there is insufficient fuel, then `sum_of_nats(N-1)` will fail, and that out-of-fuel error will be propagated by the evaluator's interpretation of `+`.

Now, we have enough machinery to to prove that if the evaluation of `sum_of_nats(N)` has enough fuel, then it returns

```
(make-erl-val-integer :val (sum-of-nats (erl-val-integer->val N)))
```

Furthermore, we show that if the fuel for the initial continuation is at least $8 * (1 + N)$, then the evaluator will not run out of fuel. From here, it is simple to show that `sum-of-nats(N)` matches the closed form formula and therefore, with sufficient fuel for the evaluator, `sum_of_nats(N)` does as well.

While there were many lemmas in the proof, most were straightforward. We expect that the lemmas that show that "translate" Erlang expressions into lisp equivalents could be automated. The main challenge here is that this translation requires the hypothesis that the evaluation does not encounter an exception or out-of-fuel condition, i.e. the result is well-formed. This is a reasonable hypothesis, as

```

reduce(none, [], GrandTotal) -> GrandTotal;
reduce(ParentPid, [], MyTotal) ->
    ParentPid ! {self(), MyTotal};
reduce(ParentPid, [ChildHd | ChildTl], LeftTotal) ->
    receive
        {ChildHd, RightTotal} ->
            reduce(ParentPid, ChildTl, LeftTotal + RightTotal)
    end.

```

Figure 5: The Erlang Reduce Implementation

we expect that verification of Erlang code will often be separated into arguments about the well-formed case and arguments about error handling. When trying to automate the well-formed case, we needed a reasoning that showed that if the result was well-formed, then so was the value of each sub-expression – until a `try-catch` expression, though there were no exception handlers in this example. The difficulty was that

- When rewriting a term, a rewrite rule can’t introduce new hypotheses about formulas produced by the rewrite rule, i.e. assertions about the subterms.
- We might hope to establish the hypotheses for rewriting the subterms by back-chaining, but this runs into a `:match-free` problem – what term is my parent?
- Forward chaining works, but for only one level of the AST. As `:doc forward-chaining` explains:

“Heuristic approval of each new addition is necessary to avoid infinite looping...”

and these heuristic block forward-chaining from propagating the well-formed hypothesis to all the nodes of the AST.

One “solution” to these problems was simply to enable a large number of rewrite rules about ASTs. This turned out to be impractical due to the many variants for each of the tag-sum types representing Erlang ASTs and values. The rewriter would generate extremely large numbers of cases and grind to a halt. We have tried some experiments using extended meta-functions with a simplified version of the evaluator. The results look very promising. We need to try this with the full evaluator.

The measure function for `sum_of_nats`, i.e. $8 * (1 + N)$ was manually derived. We believe that this could be automated in many cases.

Introducing the lisp version of `sum_of_nats` simplified reasoning tremendously. We need more examples to find out if we should generate these functions automatically, or if the need for such functions can be eliminated or greatly reduced if we provide customized simplifiers for continuations and continuation lists.

4.2 Reduce

We implemented Schwartz’s parallel algorithm for reduce [19] to compute the sum of values provided by workers each calling their local reduce function. The algorithm organizes the processes as a tree, and the grand-total is produced at the top of the tree. In Schwartz’s approach this value should be propagated back to each worker so that their call to `reduce` returns the grand total. The reduce we implement is a constrained and simplified version of Schwartz’s algorithm. It does not propagate values back to each worker.

This example is very much a demonstration that we can reason with processes. It also shows that there are many opportunities for improvement. For example, the code in

`Erlang-in-ACL2/books/reduce-v2/wtree/create-wtree/spawn.lisp` creates Erlang pids – this is part of the Erlang run-time and will be moved into the evaluator. Spawning a process requires finding an unused pid to assign to the new process. This is a globally visible action, because any other process can see that a new process has come into existence, and can send messages to it. Therefore, the Erlang `spawn` will need to pass control back to the scheduler, similar to `receive`. Otherwise, we would need to keep track of which pending messages came before a process was spawned, and which came after, to determine where to send them. The `spawn` equivalent we use in this example is an ACL2 function that creates the pids before evaluation begins, and no new processes are spawned during evaluation. For now, pids are represented as unique natural numbers.

The function `(create-wtree n)` in

`Erlang-in-ACL2/books/reduce-v2/wtree/create-wtree/create-wtree.lisp` creates a “worker tree” consisting of `n` nodes. Each node has a list of the pids for branches connected of the tree that communicate with this pid, they are ordered from the closest (and smallest) such subtree to the furthest (and largest). The function `wtree-p` in

`Erlang-in-ACL2/books/reduce-v2/wtree/wtree.lisp` is a recogniser for these worker-trees and checks for various properties that the tree should have. Each worker has a unique index; these are natural numbers from 0 to `n-1` in left-to-right order of the tree leaves. At this point in development, each worker calls the reduce function with its index, so the master process will receive the sum of all indices.

To show that the grand-total is produced at the root node, we use an invariant, see

`Erlang-in-ACL2/books/reduce-v2/proofs/inv.lisp`.

As currently written, each process terminates when it sends a value to its parent. The invariant states that the value held by the process when it reaches termination is the sum of the values at the leaves of the subtree for which the process is the root. This implies that the value of the root of the tree, the process whose index is 0, upon termination, is the sum of the values with which each process called `reduce`.

We have so far proven that if a `wtree` satisfies the invariant, then it will also be satisfied during and after evaluation. We have also shown that if the invariant is satisfied and if there are no more non-stuttering actions left for the scheduler to take, then no process in the `wtree` will be blocked (there will be no deadlocks). The next step would be to show that `reduce` will terminate, assuming the scheduler satisfies some weak fairness conditions, and thus produce the correct result in the root process.

The proof of reduce shows that we can reason about parallel computations using our continuation-passing-style evaluator. The issues with needing to guide the ACL2 rewriter carefully that we described for the `sum_of_nats` example also apply here. We believe that this can be largely automated and greatly simplified, but that remains as future work.

4.3 The Evaluator is Executable

While the scheduler is abstract, it has an executable counterpart. The executable scheduler goes through the existing processes in order until it finds a runnable process or a pending message in an outbox. This makes it possible to run and test the concurrent Erlang evaluation in ACL2. A very simple example of message passing is presented in `Erlang-in-ACL2/scheduler/examples/Joe-Robert.lisp`. In this example, two processes send messages to each other, acknowledge that they have received them, and then exit. Test cases for the reduce implementation are located in `Erlang-in-ACL2/books/reduce-v2/examples.lisp` that check the result of evaluating `wtees` of different sizes.

As the executable scheduler runs, it prints the actions taken. When complete (or out of fuel), the scheduler returns the updated list of processes. This can be very verbose, as it includes entire states of each process. For testing purposes, it is better to retrieve only the exit values of each process. Helper functions for this are included in the files,

5 Conclusions and Future Work

We have presented a continuation-passing style evaluator for Erlang in ACL2. Continuations are a natural choice for Erlang because they handle concurrency, communication, exceptions, and simple, vanilla sequential evaluation in a unified way. ACL2 is a natural choice for reasoning about Erlang because many Erlang applications have stringent reliability requirements, while dynamic updates mean the code can change after deployment. A formal verification tool for Erlang must be suitable for running regression tests in a timely manner. This is an area where ACL2 excels.

We demonstrated our evaluator by proving properties of two programs. The first was a simple, recursive function to compute the sum of the natural numbers up to N . We showed that the Erlang function computed the same value (up to mappings between data representations) as a native ACL2 implementation. From that, we showed that the value returned by the Erlang function matched a closed-form expression for the sum. The second example was a parallel reduce. We used an invariant to show that it correctly computed the sum of the values provided by each worker process.

Future Work

In this paper, we have shown that proofs are possible in this framework and that there is substantial room for improvement. For segments of code where exceptions and process preemption do not occur, reasoning is simpler when the continuation passing style is rewritten into more familiar lisp expressions. We noted technical reasons that prevent these transformations from being performed automatically by the ACL2 waterfall. We are exploring the use of meta-functions and/or clause processors to automate these transformations.

Our current scheduler only provides extremely weak fairness conditions. A weakly-fair scheduler would match the guarantees of the Erlang runtime and would allow us to reason about more realistic reactive programs.

References

- [1] ACL2 Foundation (2025): *XDOC — Defprod*. Available at https://acl2.org/doc/index.html?topic=FTY___DEFPROD.
- [2] ACL2 Foundation (2025): *XDOC — Deftagsum*. Available at https://acl2.org/doc/index.html?topic=FTY___DEFTAGSUM.
- [3] Andrew W Appel (2007): *Compiling with continuations*. Cambridge university press.
- [4] Joe Armstrong (2007): *A history of Erlang*. In: *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, pp. 6–1, doi:10.1145/1238844.123885. Available at <https://doi.org/10.1145/1238844.123885>.
- [5] Joe Armstrong (2010): *Erlang*. *Communications of the ACM* 53(9), pp. 68–75, doi:10.1145/1810891.1810910. Available at <http://doi.acm.org/10.1145/1810891.1810910>.
- [6] Joe Armstrong (2013): *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf.

- [7] Péter Bereczky, Dániel Horpácsi & Simon Thompson (2023): *A frame stack semantics for sequential Core Erlang*. In: *Proceedings of the 35th Symposium on Implementation and Application of Functional Languages*, pp. 1–13, doi:10.1145/3652561.3652566. Available at <https://doi.org/10.1145/3652561.3652566>.
- [8] Richard Carlsson (2001): *An Introduction to Core Erlang*. Colloquium on Principles, Logics, and Implementation of high-level programming languages. Available at <https://erlang.org/workshop/carlsson.ps>.
- [9] David L. Dill (1988): *Trace Theory for Automatic Hierarchical Verification of Speed-Independent Circuits*. Ph.D. thesis, School of Computer Science, Carnegie Mellon University. Published in book form as part of the ACM Doctoral Dissertation Award Series by the MIT Press, Cambridge, MA, 1989.
- [10] erlang.org: *Erlang Quick Start*. web page. Available at https://www2.erlang.org/documentation/doc-8.0/doc/getting_started/intro.html.
- [11] Lars-Åke Fredlund, Dilian Gurov, Thomas Noll, Mads Dam, Thomas Arts & Gennady Chugunov (2003): *A verification tool for Erlang*. *International Journal on Software Tools for Technology Transfer* 4(4), pp. 405–420.
- [12] Shilpi Goel, Warren A Hunt Jr & Matt Kaufmann (2017): *Engineering a formal, executable x86 ISA simulator for software verification*. In: *Provably Correct Systems*, Springer, pp. 173–209.
- [13] Fred Hébert (2013): *Learn You Some Erlang for Great Good!: a Beginner's Guide*. No Starch Press. Available at <https://learnyousomeerlang.com>.
- [14] Huiqing Li, Simon Thompson, György Orosz & Melinda Tóth (2008): *Refactoring with Wrangler, updated: Data and process refactorings, and integration with Eclipse*. In: *Proceedings of the 7th ACM SIGPLAN Workshop on Erlang*, pp. 61–72.
- [15] Tobias Lindahl & Konstantinos Sagonas (2005): *Typert: a type annotator of erlang code*. In: *Proceedings of the 2005 ACM SIGPLAN workshop on Erlang*, pp. 17–25.
- [16] Hanbing Liu & J. Strother Moore (2004): *Java Program Verification via a JVM Deep Embedding in ACL2*. In Konrad Slind, Annette Bunker & Ganesh Gopalakrishnan, editors: *Theorem Proving in Higher Order Logics*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 184–200, doi:10.1007/978-3-540-30142-4_14. Available at https://doi.org/10.1007/978-3-540-30142-4_14.
- [17] Y. Peng & M.R. Greenstreet (2019): *Verifying Timed, Asynchronous Circuits using ACL2*. In: *Proceedings of the 25th International Symposium on Asynchronous Circuits and Systems*, IEEE Computer Society, pp. 96–104, doi:10.1109/ASYNC.2019.00021. Available at <http://dx.doi.org/10.1109/ASYNC.2019.00021>.
- [18] Konstantinos Sagonas (2005): *Experience from developing the Dialyzer: A static analysis tool detecting defects in Erlang applications*. In: *Proceedings of the ACM SIGPLAN Workshop on the Evaluation of Software Defect Detection Tools*, pp. 1–5.
- [19] Jacob T Schwartz (1980): *Ultracomputers*. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 2(4), pp. 484–521.
- [20] A. Slobodova, J. Davis, S. Swords & W.A. Hunt, Jr. (2011): *A flexible formal verification framework for industrial scale validation*. In: *Formal Methods and Models for Codesign (MEMOCODE), 2011 9th IEEE/ACM International Conference on*, pp. 89–97, doi:10.1109/MEMCOD.2011.5970515. Available at <http://dx.doi.org/10.1109/MEMCOD.2011.5970515>.
- [21] Gergő Lajos Turán, Arsenii Fomin, Péter Bereczky, Dániel Horpácsi & Simon Thompson (2025): *Deriving an Erlang interpreter from a mechanised formal semantics of Core Erlang*. In: *Proceedings of the 24th ACM SIGPLAN International Workshop on Erlang*, pp. 26–39, doi:10.1145/3759161.3763046. Available at <https://doi.org/10.1145/3759161.3763046>.